

SARIMA-Based Raw-Material Demand Forecasting and Inventory Recommendation

A source-code-grounded methodology for thesis integration

Implementation examined: 8 October 2026 | Philippine calendar

Abstract

This document describes the forecasting methodology implemented in a cafe inventory management system. The system estimates store-wide daily consumption for each raw material over a seven-calendar-day period. Historical consumption records are combined with completed point-of-sale ingredient deductions, standardized to live inventory units, and aggregated into continuous daily series. A logarithmic Box-Cox transformation and a Philippine holiday indicator are incorporated into a seasonal autoregressive integrated moving-average model with an exogenous regressor. The web implementation evaluates twelve candidate specifications through chronological expanding-window validation and ranks them by average mean absolute error. Validated candidates are refitted using retained history and subjected to numerical and range checks before their predictions are accepted. Forecast totals are then compared with a live usable-stock snapshot and configured safety-stock policies to produce purchasing recommendations and stockout priorities. The document presents the implemented equations, data-processing rules, system architecture, and methodological limitations. Dataset counts describe the supplied training file; no empirical claim of predictive superiority, calibrated interval coverage, or inventory-cost reduction is made without a separate evaluation.

Keywords

SARIMA; SARIMAX; raw-material demand; inventory planning; time-series validation; point-of-sale consumption.

Use within a thesis

The numbered sections may be adapted into a methodology or system-design chapter. The appendix records implementation evidence and outstanding evaluation requirements. Institutional chapter numbering and citation formatting can be applied when integrating this material.

1. Forecasting objective and model formulation

The forecasting target is the daily quantity of an individual raw material consumed across the store. Let $y_{m,t}$ denote consumption of material m on calendar day t in its live inventory unit. Independent models are fitted for different materials. A product filter selects associated ingredients for display; it does not allocate shared ingredient demand to that product. [C1-C3]

Seasonal ARIMA represents nonseasonal dynamics with order (p,d,q) and seasonal dynamics with order $(P,D,Q)_s$. The present system sets $s = 7$ because observations are daily and include weekends. This is an imposed model period rather than evidence that every ingredient has a statistically verified weekly cycle. (Hyndman & Athanasopoulos, 2021, Section 9.9; C1)

Symbol	Meaning
p, q	Nonseasonal autoregressive and moving-average orders
d	Number of ordinary differences
P, Q	Seasonal autoregressive and moving-average orders
D	Number of seasonal differences
s	Seasonal period: seven calendar observations

The implementation uses the statsmodels SARIMAX class with a holiday regressor. Its statistical specification is therefore regression with seasonal ARIMA errors, although application metadata labels it SARIMA. The following notation suppresses the material index. (statsmodels developers, n.d.; C1)

$$z_t = \log(1 + y_t); \quad z_t = \beta h_t + u_t \quad (1)$$

$$\begin{aligned} \varphi(B)\Phi(B^7)(1-B)^d(1-B^7)^D u_t \\ = \theta(B)\Theta(B^7)\varepsilon_t \end{aligned} \quad (2)$$

Here B is the lag operator, h_t is the holiday indicator, β is its fitted coefficient, and ε_t denotes innovations. The AR and MA polynomials describe ordinary and seasonal dependence. No deterministic trend argument is supplied; the class default contains no deterministic trend term. Fitting enforces stationarity of the AR components and invertibility of the MA components. These constraints do not prove that the original consumption series is stationary. [C1]

2. Data sources and research unit

The unit of analysis is a material-day after aggregation, rather than an individual customer order. The supplied CSV contains transaction-level ingredient quantities. Direct inspection on 8 October 2026 produced the descriptive counts in Table 1. These describe the file, not the number of successful models or the current database contents. [C6]

Table 1. Descriptive scope of the supplied historical file.

Characteristic	Observed value
Filename	cafe_raw_material_daily_consumption.csv
Transaction-level rows	384,242
Earliest and latest dates	1 January 2023 - 31 July 2026
Distinct calendar dates	1,308
Distinct raw materials	55
Distinct menu products	38
Material/unit combinations	55
Units represented	L, g, kg, ml, pcs

The web worker requires date, product, raw_material, item_code, quantity_used, unit, and holiday. It parses dates, converts quantities to numeric values, and rejects nonfinite or negative usage. Other fields, such as weekday, store_status, holiday_name, and calendar descriptors, are present in the file but are not separate predictors in the configured model. [C2]

Operational history is obtained from CHECKOUT inventory ledger lines linked to orders currently marked COMPLETED. Consumption is computed from the negative quantity deductions. Recorded ledger quantities incorporate checkout modifiers and split-batch usage without reconstructing historical orders from current recipes. Waste, deliveries, and manual inventory adjustments do not enter this demand query. [C3]

Only complete Philippine calendar days are used. The cutoff is midnight at the earlier of the requested forecast start or the snapshot date in Asia/Manila. This excludes today's partial-day sales and prevents observations from the forecast period entering POS training. A separate coverage query retains dates of orders with completed timestamps, including reversed orders, so historical CSV consumption does not reappear after refunds or voids. [C3]

The repository alone does not establish whether the historical file is entirely operational, manually prepared, or simulated. A thesis should separately document its provenance, collection procedure, inclusion criteria, and authorization for research use.

3. Preparation of the daily demand series

3.1 Material identification and unit standardization

Historical materials are matched to active inventory by case-insensitive SKU, followed by case-insensitive exact name if no SKU matches. A unique match is required. Quantities are converted to the live inventory unit using predefined mass, volume, or count scales. Kilograms and grams, and litres and millilitres, differ by a factor of 1,000. Cross-dimension conversions are rejected. Bottle and piece units share a factor of one in the current conversion table; measured package-size or density conversions are not implemented. [C2]

3.2 Source precedence and aggregation

POS records replace CSV rows on POS-covered dates across matched materials; uncovered CSV dates remain historical backup. For ingredients with historical coverage, absence of consumption on a POS-covered date is represented as zero. POS-only materials begin at their first recorded consumption date. The merge prevents overlapping sources from being added together and labels retained records by source. This procedure assumes that transaction-date coverage is adequate for replacing the historical file on those dates. [C3]

$$y_{m,t} = \sum_r \mathbb{1}_{R(m,t)} a_r q_r \quad (3)$$

In Equation (3), $R(m,t)$ is the retained set of consumption records for material m on day t , q_r is recorded quantity, and a_r is its unit-conversion factor. Aggregation combines ingredient usage from all products. [C1-C3]

3.3 Calendar completion and eligibility

Every calendar date between each material's first and last retained records is included, with missing dates filled by zero. Weekends are included; the stated daily 1 PM-10 PM store schedule is contextual and is not fitted as an hourly model. The web worker requires at least sixty daily observations and rejects constant series. Filled dates count toward this minimum. Full history is used unless FORECAST_TRAINING_DAYS specifies a recent window of at least sixty days. [C1-C2]

3.4 Transformation and holiday encoding

The quantity is shifted by one and transformed using Box-Cox with fixed $\lambda = 0$, giving $z_t = \log(1 + y_t)$. This permits zeros and compresses large amounts. The function name `make_stationary` does not imply that a stationarity test is executed; no ADF or KPSS test is part of this workflow. (Hyndman & Athanasopoulos, 2021, Section 3.1; C1)

Historical holidays are flagged only when the retained value equals YES. POS flags and future flags use the Philippine holiday calendar. Filled historical dates receive a default holiday flag of zero. Consequently, historical coding and the future calendar should be checked for consistency. [C1-C3]

4. Model search and chronological validation

The application-integrated worker overrides the standalone defaults. Table 2 identifies the specifications used by the web pipeline; the thesis should name this execution path to avoid reporting settings that apply only to the standalone script. [C1-C2]

Table 2. Candidate configuration in the web worker.

Configuration	Web implementation
Nonseasonal order	$p \in \{0,1,2\}; d \in \{0,1\}; q \in \{0,1\}$
Seasonal order	$(P,D,Q,s) = (1,0,0,7)$
Candidate combinations	$3 \times 2 \times 2 = 12$
Requested validation folds	3
Selection criterion	Lowest average original-unit MAE
AR/MA restrictions	Stationarity and invertibility enforced

The standalone defaults test seventy-two combinations, fixing $d = 1$ while searching additional seasonal terms. The web pipeline instead allows either zero or one ordinary difference and keeps the seasonal specification fixed. Neither path uses an automatic ACF/PACF identification procedure. [C1-C2]

Chronological expanding-window validation trains each candidate on earlier observations and tests it on later blocks. This preserves the temporal direction of prediction. Time-series cross-validation evaluates genuine future observations relative to each training origin, rather than randomly shuffling dates. (Hyndman & Athanasopoulos, 2021, Section 5.10)

$$\text{train_end} = n - H(f + 1); \text{test_end} = \text{train_end} + H \quad (4)$$

Here n is retained series length, H is validation horizon, and f indexes requested folds. A fold is retained when it leaves at least thirty training observations. Up to three folds are used; if no rolling fold is feasible, the code falls back to a chronological 80/20 holdout. Thus, three folds are requested but are not guaranteed for every material. [C1]

The web worker sets H to the number of calendar steps from the material's last observation through the requested forecast end. H therefore includes any history gap as well as the seven displayed dates. During each fold, transformation is applied to training observations, forecasts are inverse-transformed, and errors are evaluated in original units. A candidate that raises a fitting or prediction exception in any fold is discarded. [C1-C2]

After validation, candidates are sorted by mean fold MAE. The effective selection is the first candidate in this ranking whose full-history forecast also passes the output checks described in Section 6. AIC, BIC, MAPE, and SMAPE are reported as diagnostics rather than the selection objective. [C1-C2]

5. Error measures and interpretation

Let $e_i = y_i - \hat{y}_i$ for an original-unit validation observation, and let N be the number of held-out dates in a fold. The implementation calculates the following measures. These equations describe the code's arithmetic and masking rules. [C1]

$$\text{MAE} = (1/N) \sum_{i=1..N} |e_i| \quad (5)$$

$$\text{RMSE} = \sqrt{[(1/N) \sum_{i=1..N} e_i^2]} \quad (6)$$

$$\begin{aligned} \text{MAPE} &= (100/|A|) \sum_{i \in A} |e_i/y_i|; \\ A &= \{i : y_i \neq 0\} \end{aligned} \quad (7)$$

$$\begin{aligned} \text{SMAPE} &= (100/|S|) \sum_{i \in S} 2|e_i|/(|y_i| + |\hat{y}_i|); \\ S &= \{i : |y_i| + |\hat{y}_i| \neq 0\} \end{aligned} \quad (8)$$

MAE and RMSE express error in the material's measurement unit. MAPE is undefined when all actual validation quantities are zero; the code returns NaN and the web metadata converts nonfinite values to null. SMAPE excludes zero/zero pairs and returns zero when every comparison has a zero denominator. Averaging MAPE across folds can therefore remain undefined if a fold has all-zero actual demand. [C1-C2]

$$\text{Score}(c) = (1/K) \sum_{k=1..K} \text{MAE}_{c,k} \quad (9)$$

In Equation (9), c identifies a candidate and K is the number of actual validation folds. MAE remains defined on zero-demand days, which is why it is used for candidate selection. Metrics are averaged across folds; saved AIC and BIC describe those validation fits rather than being recomputed from the final refit. [C1]

Percentage errors require particular care when actual consumption is zero or near zero, and accuracy measures based on historical evaluation do not establish future accuracy. Comparisons across ingredients should acknowledge that absolute errors have different units and demand scales. (Hyndman & Athanasopoulos, 2021, Section 5.8)

The standalone evaluation display also computes $\max(0, 100 - \text{MAPE})$ and labels it Confidence. This is a heuristic complement of error, not a statistical confidence level or probability of a correct forecast. It should not be reported as a thesis accuracy guarantee. [C1]

6. Forecast generation and output acceptance

Each validated candidate is refitted on all retained transformed observations with the holiday regressor. Prediction begins on the day after that material's last observation. Future Philippine holiday flags are supplied for the entire horizon. The seven requested dates are selected from these predictions. The start date must follow the combined history's latest date; otherwise the worker rejects the request. [C1-C2]

$$\hat{y}_{T+h} = \max(0, \exp(\hat{z}_{T+h}) - 1) \quad (10)$$

Equation (10) expresses the fixed-lambda inverse transformation before rounding to two decimals. The software also inverse-transforms and floors the lower and upper nominal 95% bounds at zero. Back-transformed point predictions receive no explicit log-scale bias adjustment; therefore they should not automatically be interpreted as arithmetic conditional means on the quantity scale. [C1]

The nominal interval is obtained through `get_forecast()` and `conf_int(alpha = 0.05)`. Its width reflects model-based uncertainty. Interval coverage depends on modeling assumptions and must be evaluated empirically; a 95% label does not mean that the point forecast is 95% accurate. (Hyndman & Athanasopoulos, 2021, Section 9.8; C1)

$$C_{\max} = \min(10^{14}, 100 \times \max(1, \max_t y_t)) \quad (11)$$

For each candidate in MAE order, the bridge checks the seven output estimates and both bounds against Equation (11). All returned values must be finite, nonnegative, and strictly below the ceiling. Bounds are adjusted to contain the point estimate. An excessive positive forecast is rejected rather than capped into an acceptable value. However, the underlying model routine already clips negative inverse-transformed quantities at zero. [C1-C2]

If a candidate fails, the worker tries the next validated specification. If no candidate passes, the material is excluded with a warning. A run can still complete with other materials; if no material succeeds, it fails. The guard screens numerical and extreme-range outputs without imposing a maximum acceptable validation error. Passing it is therefore not evidence of predictive quality. [C2]

The default permitted distance from the combined latest history date to forecast start is 365 days, configurable through `FORECAST_MAX_HISTORY_GAP_DAYS`. A distance over thirty days generates a stale-history note. Ingredient-specific history can be older than the combined latest date, and its own prediction gap is disclosed in the audit. Unobserved gap dates are estimated rather than supplied as observed demand. [C2]

7. Forecast-driven inventory recommendation

Demand prediction is followed by a deterministic inventory interpretation. Let F be total forecast demand, N the number of forecast dates, C current usable stock, S safety stock, and L lead time in days. Stock originates from the live inventory snapshot. The web path takes S and L from a uniquely matching `current_inventory.csv` policy and converts S into live units; CSV `CurrentStock` values are ignored. Missing policy values default to zero. [C2, C4]

$$F = \sum_{h=1..N} \hat{y}_h; \quad a = F/N \quad (12)$$

$$\text{Lead-time demand} = aL; \quad \text{Reorder point} = aL + S \quad (13)$$

$$\text{Stock coverage} = C/a \quad (a > 0) \quad (14)$$

$$\text{Recommended purchase } Q = \text{ceil}[\max(0, F + S - C)] \quad (15)$$

The code rounds a to two decimals before using it in lead-time demand and stock coverage. The purchase formula uses F directly and rounds up to a whole inventory unit. Coverage is infinite when average demand is zero; it is represented as null in JSON output. Missing live stock produces No Data and disables the actionable purchase quantity. [C2, C4]

Stockout timing is computed by subtracting each daily forecast from the snapshot stock in date order and finding the first one-based date index at which remaining stock is at or below zero. It differs from average stock coverage because it uses the daily demand pattern. [C4]

Table 3. Implemented purchase-priority rules.

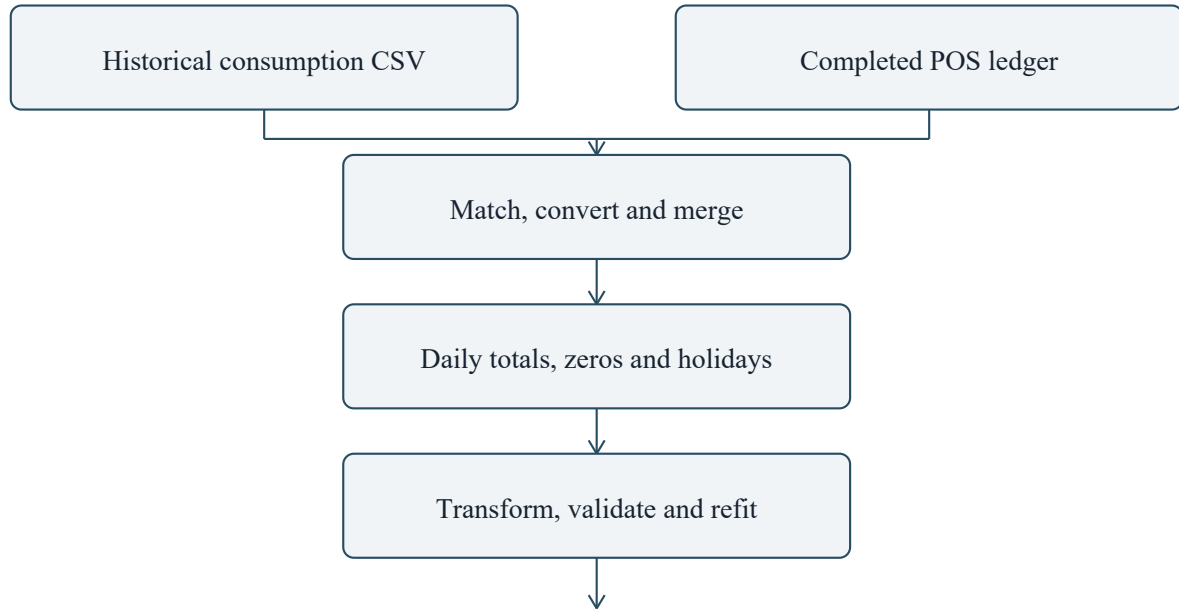
Condition evaluated in order	Label
Inventory record is missing	No Data
Recommended purchase is zero	Healthy
Purchase needed; no stockout within the forecast	Low
First stockout on day 1 or 2	Critical
First stockout on day 3 or 4	High
Later stockout within the forecast	Medium

For illustration only, seven forecasts of 10 kg give $F = 70$ kg. With $C = 30$ kg, $S = 10$ kg, and $L = 2$ days, average demand is 10 kg/day, reorder point is 30 kg, coverage is three days, and $Q = 50$ kg. The simulated stockout is day three and priority is High. These are hypothetical values, not fitted results.

The purchase trigger is Equation (15), not the reported reorder point. Lead time does not alter that purchase formula or the fixed priority thresholds. The database material reorder point is passed to the worker but is not used by this calculation. [C2, C4-C5]

8. Architecture and operational integration

The forecasting feature connects a NestJS backend, a noninteractive Python worker, PostgreSQL storage, and a React/Next.js interface. Figure 1 summarizes the implemented demand-processing sequence. Live stock and CSV safety-stock policies join the pipeline when accepted predictions are converted into recommendations. [C1-C5]



Seven daily forecasts -> stock recommendations -> saved results
Figure 1. Implemented forecasting and recommendation workflow.

The service checks scheduling on startup and every sixty seconds while the backend is running. A new seven-day period starts once the latest completed period has ended in Philippine time. After downtime it starts on the current date instead of backfilling past predictions. A failure within the preceding hour delays retry. Individual checkouts do not retrain the model. [C5]

POS consumption and usable stock are read within one repeatable-read database transaction. An active-run key and a PostgreSQL advisory lock serialize run claims. The service writes a temporary JSON request and spawns the Python process without a shell, with a thirty-minute time limit. The worker returns predictions, recommendations, source identifiers, and audit metadata. [C5]

Before persistence, the backend requires exactly seven expected consecutive dates per unique material, finite nonnegative estimates below the storage threshold, and bounds that contain the estimate. Unknown material identifiers are rejected. Completion and all material results are saved transactionally; late workers cannot replace terminal run status. Temporary files are removed after execution. [C5]

Stored run, series, point, and recommendation records retain model orders, error measures, training dates, source counts, filled dates, gap lengths, validation windows, exclusions, and stock-based advice. Trained Python objects are not serialized. The interface supports saved-period comparison and product/ingredient filtering. Refreshing the page reloads saved results; it does not initiate model fitting. [C5]

9. Scope, assumptions, and limitations

The implemented target is observed ingredient consumption from completed sales. It is not unconstrained customer demand: unavailable items or stockouts may suppress recorded consumption. Independent material models also do not enforce recipe-level coherence across ingredients. These distinctions should be stated when interpreting the system's output.

Zero filling assumes that absent daily records represent no consumption. A missing or incomplete record can instead indicate an unobserved trading day. Since filled dates count toward history eligibility, sixty observations do not necessarily mean sixty measured sales days. POS replacement likewise assumes sufficient coverage of store transactions. [C1-C3]

Seven-day seasonality is configured for all materials. The code does not execute stationarity tests, residual whiteness tests, baseline comparisons, or explicit optimizer-convergence rejection. Warnings are globally suppressed in SARIMA.py. A successful fit and plausible range are insufficient to establish model adequacy. [C1-C2]

Forecasts use a holiday indicator but no configured weather, price, promotion, event, traffic, or recipe-change predictors. The historical and future holiday encodings may differ. Long gaps and operational changes can weaken the relevance of older history; the maximum-gap setting only permits extrapolation and does not add measured observations. [C1-C3]

The inventory layer uses a saved usable-stock snapshot. Existing recommendations do not update when stock changes. Purchase quantities do not account for delivery arrival times, supplier package sizes, expiry, incoming orders, or stock consumed before a future forecast start. Safety stock is configured externally rather than estimated from forecast uncertainty. Purchasing uses point forecasts rather than upper interval bounds. [C2, C4-C5]

MAE-based selection can choose a model with large residual error because no acceptance threshold is configured. MAPE excludes zero actual values, making its interpretation incomplete for intermittent demand. Back-transformed estimates lack explicit bias adjustment, and the nominal interval coverage has not been established for this dataset. [C1-C2]

The source hash combines CSV bytes with POS consumption and coverage JSON; it omits model code, settings, stock, and policy inputs. It is therefore a partial source identifier rather than a complete experiment fingerprint. Reproducible thesis experiments should record the source revision, dependency versions, all policies and configuration, data snapshots, and retained evaluation windows. [C2]

10. Evidence and proposed empirical evaluation

This document establishes what the inspected software implements. It does not establish that SARIMA outperforms alternatives or reduces inventory costs. The dataset counts are descriptive observations. No full fifty-five-material fitting run, independent final holdout evaluation, calibrated interval test, or live database experiment was performed for this document.

10.1 Evaluation design for a results chapter

A separate experiment should freeze eligible source records and unit policies, designate a final chronological test period, and perform parameter selection using only earlier observations. Selection validation and final testing should be distinguished so that the same held-out windows do not serve as both the selection evidence and the final claim of generalization. This is a proposed research procedure, not an additional feature already implemented.

At each evaluation origin, models should produce seven calendar-day forecasts using information available before that origin. Comparators can include previous-day demand and a seasonal naive forecast that repeats the previous week. These baselines and any additional model families must be fitted and scored under identical data availability and preprocessing rules. When histories contain gaps, both gap length and forecast step should be reported.

The results chapter should report material-level MAE and RMSE, fold variability, zero-demand frequency, sample counts, candidate exclusions, and runtime. Scale-dependent errors should be reported in their original units; summaries across heterogeneous materials require an explicitly defined normalization or weighting procedure. MAPE and SMAPE should use the exact masking rules in Section 5.

Interval evaluation should measure empirical coverage and width for the nominal 95% range. Inventory simulation should separately specify delivery lead times, replenishment rules, expiry handling, and stockout definitions before claiming operational improvement. Sensitivity experiments may compare zero filling against better documented missing-data treatment, full history against recent windows, and models with and without the holiday feature.

10.2 Reporting boundary

A defensible conclusion from implementation inspection is that the application supplies a traceable seven-day forecasting and stock-planning workflow. Claims about forecast accuracy, superior model choice, service-level improvement, or purchasing savings require measured results. Comments in the source asserting higher holiday demand are not substituted for a reported statistical analysis.

Appendix A. Source-code evidence and execution paths

The identifiers below are local implementation sources inspected on 8 October 2026. Function and class names identify the reviewed behavior independently of line-number shifts. These sources support the method description; the external references support statistical terminology.

ID	Source and principal evidence
C1	python/SARIMA.py: CONFIG; aggregate_daily_sales; build_daily_series; make_stationary; train_sarima; fit_full_model; forecast_future; error helpers.
C2	python/forecast_bridge.py: conversion; match_material; generate; select_stable_forecast; training_audit.
C3	python/pos_history.py: merge_pos_history. ims-backend/src/forecasting/pos-history.ts: loadPosHistory.
C4	python/InventoryRecommendation.py: create_forecast_summary; simulate_inventory; build_report.
C5	ims-backend/src/forecasting/forecasting.service.ts and forecasting.types.ts; ims-backend/prisma/schema.prisma; frontend forecasting feature, graph, notes, and API helpers.
C6	python/cafe_raw_material_daily_consumption.csv: direct row, date, material, product, and unit counts.

The web worker is the methodology's primary execution path. Standalone SARIMA uses CSV alone, defaults to seventy-two model specifications, and forecasts immediately after the file's latest date. It does not merge live POS. With the supplied CSV, that standalone period is 1-7 August 2026 regardless of a later execution date. Its optional --schedule mode repeats after a seven-day sleep while the process remains running. [C1]

The standalone inventory entry point references TOP10_FILE without defining it. It can write its first two reports before failing at the top-ten export. The web worker imports build_report and does not execute that entry point. This defect does not alter the described web recommendation formula but limits standalone reproducibility. [C4]

Verification on 8 October 2026: all seventeen Python tests passed. Twenty-two backend tests passed across scheduling, POS history, and output validation. Fourteen HTTP authorization tests could not connect to localhost because of sandbox EACCES restrictions; they are not reported as passing or as established implementation defects. Several fitting tests use mocks, so these checks are software verification rather than empirical forecast-accuracy results. No live database evaluation was performed.

Historical file SHA-256:
35b622f6f09b22f92e5fab401c0442b35356517f31acafc2fbf8bb5adc5c1703

References

The statistical discussion is paraphrased from the primary sources below. Each URL was accessed on 8 October 2026. Section-specific links identify the material supporting seasonal specification, transformation, validation, error measures, and prediction intervals.

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. Section 9.9: Seasonal ARIMA models.

<https://otexts.com/fpp3/seasonal-arma.html>

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. Section 3.1: Transformations and adjustments.

<https://otexts.com/fpp3/transformations.html>

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. Section 5.10: Time series cross-validation.

<https://otexts.com/fpp3/tscv.html>

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. Section 5.8: Evaluating point forecast accuracy.

<https://otexts.com/fpp3/accuracy.html>

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. Section 9.8: Forecasting.

<https://otexts.com/fpp3/arma-forecasting.html>

statsmodels developers. (n.d.). SARIMAX API documentation, version 0.14.4. Version-specific reference consistent with the repository requirement $\text{statsmodels} \geq 0.14, < 0.15$.

<https://www.statsmodels.org/v0.14.4/generated/statsmodels.tsa.statespace.sarimax.SARIMAX.html>

Local implementation and dataset

Salvacion inventory management system. (2026). *Forecasting implementation and supplied consumption dataset* [Unpublished local source code and CSV]. Inspected 8 October 2026. Components are identified as C1-C6 in Appendix A. Dataset provenance must be documented by the researcher; repository presence alone does not establish its collection origin.

Document scope: methodology and system-design draft based on inspected implementation. Hypothetical arithmetic examples and proposed evaluation procedures are explicitly separated from observed dataset counts and implemented software behavior.